cursor-vibe-starter: An Auditable Workspace for AI-Directed Security Engineering

Ali Murtaza Bhutto
Independent Researcher
ORCID: 0009-0007-2787-943X
github.com/thunderstornX/cursor-vibe-starter

***Abstract*—"Vibe coding" — the increasingly common practice of writing software with an inline AI assistant in the loop — has so far been discussed primarily through productivity claims ("I shipped $N\times$ faster"). This paper argues for a different framing: the artefact left behind *after* an AI-directed session ought to include the rules the model was working under and the prompts the human used, and the resulting codebase ought to be tagged at the commit level so the share of AI-versus-human work can be derived from the git log without resorting to style heuristics. We present `cursor-vibe-starter`, an open-source reference workspace built on this framing. The workspace ships a working `.cursorrules` configuration, twelve five-part prompt templates for security-engineering tasks, a complete worked example (a FastAPI security microservice with a 20-test pytest suite), and a single-file LOC analysis script that reports per-tag commit breakdowns from the repository's own git log. We discuss why we deliberately avoid productivity claims and describe the trade-offs of the author-tagged measurement protocol.**

***Index Terms*—AI-directed development, prompt engineering, Cursor IDE, software methodology, security engineering, FastAPI**

I. Introduction

A pattern has emerged in 2025–2026 software engineering: a developer sits in front of a code editor with an inline LLM assistant, gives it short natural-language instructions, reviews the resulting patches, and ships the diff. Adoption is uneven; published methodology is more uneven still. The literature on this practice is dominated by productivity claims ("$N\times$ faster", "$M\%$ of my code is now AI-written") that are typically self-reported, rarely controlled, and almost never reproducible.

This paper takes a smaller bite. We do not measure speedup. We propose — and ship working artefacts for — a discipline under which a codebase produced this way is itself a record of how it was produced:

- The IDE-level rules the model was working under are committed to the repository (`.cursorrules`).
- The prompt library the human used is committed to the repository, with each prompt's pitfalls section drawn from real "the model produced this and I had to fix it" experience.
- Every commit in the repository is tagged at message level (`[ai]`, `[manual]`, `[mixed]`, `[tooling]`, `[merge]`); a single-file script aggregates these into a public LOC table.

Contributions. (i) An opinionated, open-source `.cursorrules` for security-engineering work; (ii) a library of twelve prompt templates with a strict five-part structure; (iii) a complete worked-example FastAPI microservice exercising JWT auth, Redis-backed rate limiting, and structured logging, with a 20-test pytest suite that all pass; (iv) a measurement protocol based on author-tagged commits, with the rationale for why we avoid style-based AI-detection heuristics.

Fig. 1. The repository banner. "Vibe coding" is the input; the audit-friendly artefact-with-protocol is the output.

II. Workspace Architecture

The repository has three top-level deliverables:

A. The `.cursorrules` file

A single Markdown file in the repo root that the Cursor IDE feeds to the model on every session. It encodes the project's stance on: **style** (Python 3.10+, type hints everywhere, comments explain "why" not "what"); **security** (no hardcoded secrets, parameterised SQL, JWT `algorithms=[ALG]` as a list to defeat alg-confusion, never log a request body); **testing** (every public function gets a pytest test, mock the wire not the function under test); **frameworks** (FastAPI, pydantic v2, httpx, pytest); and **response format** (lead with the change, flag security-relevant decisions, no marketing language).

B. The prompt library

Twelve numbered Markdown files, each a copy-pasteable template covering a discrete security-engineering task: API design, security review, test generation, container packaging, schema design, JWT auth, error handling, structured logging, CI pipeline, maintainability refactor, documentation, and incident-response tooling.

Every template enforces the same five-part structure:

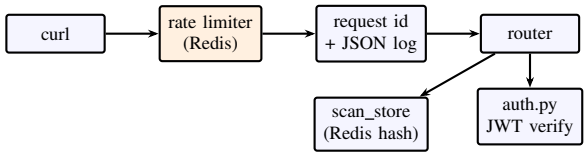

Fig. 2. Request lifecycle through the worked-example service. Outer-most middleware is rate limiting; auth is a route-level dependency so unauth'd `/health` stays cheap.

TABLE I
WORKED-EXAMPLE ENDPOINTS AND THEIR STABLE ERROR CODES.

Method	Path	Auth	Error codes
GET	/health	none	—
POST	/v1/auth/login	none	auth.bad_credentials
POST	/v1/scans	Bearer	request.invalid, rate.limited
GET	/v1/scans/{id}	Bearer	scan.not_found
GET	/v1/scans	Bearer	—

1) **Context** — when this prompt is the right one to reach for.
2) **Template** — the actual prompt with {`placeholders`} for project-specific details.
3) **Example** — a filled-in example showing real usage.
4) **Expected Output** — what good output looks like.
5) **Common Pitfalls** — failure modes drawn from real production experience with the prompt.

Crucially, the *Common Pitfalls* section is what differentiates this library from a generic prompt collection. Each entry there is a documented model failure. For example, `prompts/06_auth_flow.md`'s pitfalls section explicitly warns about `algorithms=ALGORITHM` (a string) versus `algorithms=[ALGORITHM]` (a list) — because PyJWT silently accepts *any* algorithm if the parameter is a string, which is the canonical JWT alg-confusion exploit.

C. The worked example

`example/security-microservice/` is a complete FastAPI service whose architecture is summarised in Fig. 2. Twenty pytest cases cover the auth path, scan endpoints, rate limiter, log redaction, and health probe. The full suite runs in ∼3 seconds.

III. Measurement Protocol

We claim no productivity multiplier. We do propose a transparent, gameable, but *honest* record of which lines in a release came through which workflow.

Each commit's first line begins with one of five tags: `[ai]` (AI-directed: prompt → patch → review), `[manual]` (human-typed), `[mixed]` (collaboration where neither dominates), `[tooling]` (generated configs, lockfiles, CI yaml; bucketed separately because LOC ratios from generated text say nothing about effort), or `[merge]` (merge commit with no original content; excluded from totals). Untagged commits land in an `[untagged]` bucket that exists to make discipline drift visible rather than silently inflating one of the other buckets.

`metrics/loc_analysis.py` parses the repository's `git log`, computes per-commit `insertions − deletions` over tracked source files (excluding vendored, generated, and binary paths), and aggregates by tag. Output formats: text table, GitHub-flavoured markdown, JSON.

A. Why no AI-style heuristics?

It is tempting to write a clever script that infers "AI lines" from docstring length, type-hint density, or naming patterns. We considered and rejected this approach for three reasons:

1) **Circularity.** The most reliable AI-style fingerprints are those of the *current* model generation; they break the moment a different model with a different style is used.
2) **Gaming.** A heuristic that flags "AI" on long docstrings rewards rewriting docstrings, which is busywork that doesn't change the underlying question.
3) **Hidden tail.** Real AI-pair-coded codebases contain a lot of short functions and one-line wires — exactly the cases heuristics miscount. The honest answer is "the author tells you".

The author-tagged protocol is the same trade-off used in academic authorship lists: contributors say what they contributed, and the report aggregates. If the author tags wrong, the report is wrong; that is a feature, because it forces honesty rather than rewarding clever metric-gaming.

IV. Limitations

No effort tracking. A 50-line `[ai]` patch that took two hours of human review counts identically to one the human waved through. The metric measures *output*, not *effort*.

Autocomplete blur. Modern editors complete tokens and short lines automatically. The threshold for `[ai]` in this protocol is "the patch was written by the model, not the cursor". That is judgment, not measurement.

Refactor invisibility. A `[manual]` refactor that nets to zero LOC reports zero. Refactors should be inspected separately if effort is the question.

Single-author bias. The protocol works as written for a single-author repo or a small team with shared discipline. A larger team would need a code-review-time tag-enforcement step or the discipline drifts.

V. Reproducibility

The repository is released under the MIT licence. The `.cursorrules` file, the twelve prompts, and the worked-example service are deposited at every release tag; a fresh clone will reproduce all 20 pytest cases in ∼3 seconds and all four DevSecOps gates (Bandit medium-or-higher, pip-audit, Semgrep `p/python` + `p/security-audit`, plus no-AI-attribution-trace audit) clean. The DOI placeholder is `10.5281/zenodo.20480451`.

Acknowledgement

This work cites and complements the OSINT-legal framework in [2], applied here to the question of recording and disclosing AI involvement in shipped software.

References

[1] Cursor, "Cursor IDE Documentation," https://cursor.sh/docs, 2024 (accessed 2026).
[2] A. M. Bhutto, "Operationalising the OSINT Lifecycle: A Framework for Privacy-Aware and Compliant OSINT Operations," Zenodo, 2025. doi:10.5281/zenodo.16924934
[3] OWASP, "Top 10 for Large Language Model Applications," 2025 revision, https://owasp.org/www-project-top-10-for-large-language-model-applications/.
[4] J. Padilla et al., "PyJWT," https://pyjwt.readthedocs.io, 2024 (accessed 2026).

AMB · ORCID 0009-0007-2787-943X · v1.0 · 2026